UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

I. PORTADA

Tema: CATÁLOGO DE LA BIBLIOTECA FISEI CON ÁRBOL GENERAL Y ABB EN JAVA

Unidad de Organización Curricular: BÁSICA

Nivel y Paralelo: 3 - B

Alumnos participantes: Altamirano Segovia Jullisa Brigitte

Asignatura: Estructura de datos

Docente: Ing. José Caiza

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Implementar en Java un árbol general n-ario y un Árbol Binario de Búsqueda (ABB) mediante programación orientada a objetos y recursividad, aplicando operaciones de inserción, búsqueda, recorridos, eliminación y análisis de su estructura, utilizando como temática el catálogo de la Biblioteca FISEI.

Específicos:

• Modelar la jerarquía BIBLIOTECA → AREA → ESTANTE mediante enumeraciones y clases con validaciones.

• Implementar el ABB con inserción, búsqueda, recorridos (preorden, inorden, posorden) con su traza, modificación y eliminación de sus tres casos (hoja, un hijo y dos hijos con sucesor inorden).

• Resolver los retos: contarEnRango con poda, caminoBusqueda, esABBValido con límites y el extra desdePreorden.

• Ejecutar la batería de pruebas (mínimo 10 casos), resolver a mano TRAZAS.md y documentar el repositorio en GitHub.

2.2 Modalidad

Individual

2.3 Tiempo de duración

Presenciales: 6 No presenciales: 0

2.4 Instrucciones

Construya una aplicación Java para el catálogo de la Biblioteca FISEI: la Biblioteca se organiza en Áreas y cada Área en Estantes (árbol general n-ario); cada Estante almacena sus libros en un ABB ordenado por el código del libro (entero positivo, sin repetidos).

• Parte A: enumeraciones y modelo (TipoUnidad, TipoRecorrido, Libro, UnidadBiblioteca, NodoUnidad, NodoLibro).

• Parte B: clase ArbolBiblioteca (agregar, buscar, altura, contarHojas, nivel, mostrarJerarquia, contarLibrosTotal).

• Parte C: clase CatalogoABB (insertar, buscar, contar, recorrer, trazaRecorrido, modificar, eliminar, getRaiz/setRaiz).

• Parte D: retos (contarEnRango con poda, caminoBusqueda, esABBValido, extra desdePreorden).

• Parte E: menú interactivo con Scanner y PruebasBiblioteca con mínimo 10 casos mostrando X/N pruebas OK.

• Casos de prueba obligatorios: altura 0/1 hoja con solo raíz; biblioteca inicial altura 2, 4 hojas, 13 libros; niveles 0, 1, 2 y -1; estante como hijo de BIBLIOTECA = false; recorridos de SW-E1; código repetido = false; caminoBusqueda(42) = [60, 35, 45, 40]; contarEnRango(40, 75) = 5; insertar 42 y eliminar 20, 70 y 60 con preorden final 75 35 45 40 42 80 90; códigos inexistentes = null/false; IllegalArgumentException con datos inválidos; esABBValido() = false con un nieto mal ubicado.

• Restricciones: no usar TreeSet, TreeMap, PriorityQueue, NavigableSet, NavigableMap, Collections.sort ni List.sort; todas las operaciones mediante recursividad.

• Entrega: repositorio público Deber-Arboles-ApellidoNombre, rama desarrollo unida a main con pull request, mínimo tres commits, código, README.md y TRAZAS.md.

2.5 Listado de equipos, materiales y recursos

Computadora. Material didáctico. Plataforma de diseño de UML: draw.io. Inteligencia Artificial. Lenguaje de programación Java. Entorno de desarrollo Visual Studio Code. Sistema de control de versiones Git y GitHub.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☐ Plataformas educativas ☐ Simuladores y laboratorios virtuales ☐ Aplicaciones educativas

☒ Recursos audiovisuales ☐ Gamificación ☒ Inteligencia Artificial

2.6 Actividades por desarrollar

1. Ejercicio

Construya el catálogo de la Biblioteca FISEI con árbol general y ABB. Estructura inicial (los libros se insertan en el orden indicado):

```
BIB Biblioteca FISEI
 ├── SW   Software
 │    ├── SW-E1 Estante 1 de Software   →  60, 35, 80, 20, 45, 70, 90, 40, 75
 │    └── SW-E2 Estante 2 de Software   →  15, 10, 25
 ├── TI   Tecnologías de la Información
 │    └── TI-E1 Estante 1 de TI         →  50
 └── TEL Telecomunicaciones
```

La biblioteca contiene 7 unidades, 4 hojas, una altura de 2 aristas y 13 libros.

2. Solución propuesta

Se crean dos estructuras: un árbol general n-ario (ArbolBiblioteca con NodoUnidad) que representa la jerarquía de unidades de la biblioteca, y un ABB (CatalogoABB con NodoLibro) por cada estante, que administra los libros ordenados por código. Todo se implementa con recursividad y programación orientada a objetos, organizado en los paquetes: enums (TipoUnidad, TipoRecorrido), modelo (Libro, UnidadBiblioteca), negocio (NodoUnidad, NodoLibro, ArbolBiblioteca, CatalogoABB) y app (Main, DatosPrueba, PruebasBiblioteca).

3. Especificación de las estructuras (valores y operaciones)

ArbolBiblioteca

Valores: NodoUnidad (raiz).

Operaciones: agregar : ArbolBiblioteca x String x UnidadBiblioteca --> Booleano; buscar : ArbolBiblioteca x String --> NodoUnidad; altura : ArbolBiblioteca --> int; contarHojas : ArbolBiblioteca --> int; nivel : ArbolBiblioteca x String --> int; mostrarJerarquia : ArbolBiblioteca --> String; contarLibrosTotal : ArbolBiblioteca --> int.

CatalogoABB

Valores: NodoLibro (raiz).

Operaciones: insertar : CatalogoABB x Libro --> Booleano; buscar : CatalogoABB x int --> Libro; contar : CatalogoABB --> int; recorrer : CatalogoABB x TipoRecorrido --> List<Libro>; trazaRecorrido : CatalogoABB x TipoRecorrido --> String; modificar : CatalogoABB x int x String x String x int --> Booleano; eliminar : CatalogoABB x int --> Booleano; contarEnRango : CatalogoABB x int x int --> int; caminoBusqueda : CatalogoABB x int --> List<Integer>; esABBValido : CatalogoABB --> Booleano; desdePreorden : int[] --> CatalogoABB.

4. Relación entre requisitos y código

• Jerarquía BIBLIOTECA → AREA → ESTANTE: método puedeTenerHijo de TipoUnidad y validación en ArbolBiblioteca.agregar().

• Códigos sin repetidos y búsqueda sin mayúsculas: existeCodigo() y buscar() con equalsIgnoreCase().

• Traza con formato "entra X / VISITA X / null -> regresa": CatalogoABB.trazaRecorrido().

• Eliminación de tres casos: CatalogoABB.eliminar() (hoja, un hijo, dos hijos con sucesor inorden).

• Retos con poda y límites: contarEnRango(), caminoBusqueda(), esABBValido(), desdePreorden().

• Menú y pruebas: app/Main.java, app/PruebasBiblioteca.java.

5. Paso 1: crear las enumeraciones y el modelo

TipoUnidad define puedeTenerHijo() con la regla de la jerarquía; Libro valida código > 0 (sin setter), título, autor y año 1900–2026 lanzando IllegalArgumentException; UnidadBiblioteca valida que el código esté en mayúsculas.

```
public boolean puedeTenerHijo(TipoUnidad hijo) {
    switch (this) {
        case BIBLIOTECA: return hijo == AREA;
        case AREA:       return hijo == ESTANTE;
        default:         return false; // un ESTANTE no puede tener hijos
    }
}
```

6. Paso 2: implementar el árbol general (ArbolBiblioteca)

NodoUnidad guarda la unidad, la lista de hijos y un CatalogoABB solo si la unidad es ESTANTE. Todas las operaciones (agregar, buscar, altura, contarHojas, nivel, mostrarJerarquia, contarLibrosTotal) recorren el árbol de forma recursiva. La altura se mide en aristas: un árbol con solo raíz tiene altura 0.

7. Paso 3: implementar el ABB (CatalogoABB)

La inserción baja recursivamente comparando códigos; la búsqueda es O(h); los recorridos acumulan la secuencia según el tipo; la traza imprime "entra X" al entrar al nodo, "VISITA X" al procesarlo y "null -> regresa" en un hijo vacío, con 2 espacios por nivel y la última línea "Secuencia TIPO: ...". La eliminación cubre los tres casos y usa el sucesor inorden (mínimo del subárbol derecho) para nodos con dos hijos.

8. Paso 4: ejecutar el menú y los casos de prueba

El menú de Main permite cargar la estructura inicial, mostrar la jerarquía, agregar/buscar unidades, consultar altura/hojas/nivel/total de libros, administrar libros de un estante, mostrar recorridos con traza y ejecutar los retos. PruebasBiblioteca ejecuta 25 verificaciones de los casos obligatorios.

9. Paso 5: mostrar los resultados

Ejecución real del programa (java -cp bin app.PruebasBiblioteca). Resultado final:

```
==================================================
25/25 pruebas OK
==================================================
```

10. Tipos de datos utilizados

enum TipoUnidad { BIBLIOTECA, AREA, ESTANTE }; enum TipoRecorrido { PREORDEN, INORDEN, POSORDEN };

Libro { int codigo (final, sin setter); String titulo; String autor; int anio };

UnidadBiblioteca { String codigo; String nombre; TipoUnidad tipo };

NodoUnidad { UnidadBiblioteca dato; List<NodoUnidad> hijos; CatalogoABB catalogo };

NodoLibro { Libro dato; NodoLibro izquierdo, derecho };

11. Diagrama de clases

```
TipoUnidad ......... BIBLIOTECA, AREA, ESTANTE; +puedeTenerHijo(TipoUnidad): boolean
TipoRecorrido ..... PREORDEN, INORDEN, POSORDEN
UnidadBiblioteca .. -codigo: String  -nombre: String  -tipo: TipoUnidad
Libro ............. -codigo: int (final)  -titulo: String  -autor: String  -anio: int
NodoUnidad ........ -dato: UnidadBiblioteca  -hijos: List<NodoUnidad>  -catalogo:
CatalogoABB (solo ESTANTE)
NodoLibro ......... -dato: Libro  -izquierdo: NodoLibro  -derecho: NodoLibro
ArbolBiblioteca ... -raiz: NodoUnidad | +agregar +buscar +altura +contarHojas +nivel
+mostrarJerarquia +contarLibrosTotal
CatalogoABB ....... -raiz: NodoLibro | +insertar +buscar +contar +recorrer
+trazaRecorrido +modificar +eliminar +contarEnRango +caminoBusqueda +esABBValido
+desdePreorden
Main / DatosPrueba / PruebasBiblioteca ... app
```

12. Especificación: invariantes

• Todo hijo de un ESTANTE es imposible: solo BIBLIOTECA puede tener AREA y solo AREA puede tener ESTANTE.

• No existen dos unidades con el mismo código (mayúsculas/minúsculas equivalentes) en el árbol general.

• En cada ABB no existen dos libros con el mismo código y para todo nodo: izquierdo < nodo < derecho.

• Libro impone codigo > 0 y 1900 ≤ anio ≤ 2026; el código es inmutable (no existe setter).

• buscar(), recorrer() y trazaRecorrido() nunca modifican el estado de las estructuras.

13. Pseudocódigo

```
INSERTAR(nodo, libro):
    si nodo = null: devolver nuevo NodoLibro(libro)
    si libro.codigo < nodo.dato.codigo: nodo.izquierdo = INSERTAR(nodo.izquierdo, libro)
    si no:                              nodo.derecho   = INSERTAR(nodo.derecho, libro)
    devolver nodo


ELIMINAR(nodo, codigo):
    si codigo < nodo.dato.codigo:  nodo.izquierdo = ELIMINAR(nodo.izquierdo, codigo)
    si codigo > nodo.dato.codigo:  nodo.derecho   = ELIMINAR(nodo.derecho, codigo)
    si no:
        si nodo no tiene hijos: devolver null                      // hoja
        si nodo tiene un solo hijo: devolver ese hijo              // un hijo
        sucesor = MINIMO(nodo.derecho)                             // dos hijos
        nodo.dato = sucesor.dato
        nodo.derecho = ELIMINAR(nodo.derecho, sucesor.dato.codigo)
    devolver nodo
```

14. Prueba de escritorio: trazas reales del programa

Salida verificada con la ejecución del programa (secuencia de SW-E1 + los 3 códigos personales 28, 87, 78, derivados de la cédula 1805262878):

```
=== 1. Secuencia insertada en SW-E1 ===
60, 35, 80, 20, 45, 70, 90, 40, 75
Preorden: 60 35 20 45 40 80 70 75 90
Inorden:  20 35 40 45 60 70 75 80 90
Posorden: 20 40 45 35 75 70 90 80 60


=== 2. Insercion de los 3 codigos personales: 28, 87, 78 ===
Preorden tras insertar: 60 35 20 28 45 40 80 70 75 78 90 87
Inorden tras insertar:  20 28 35 40 45 60 70 75 78 80 87 90
```

```
Posorden tras insertar: 28 20 40 45 35 78 75 70 87 90 80 60
```

Traza del recorrido en PREORDEN (2 espacios por nivel):

```
entra 60
VISITA 60
  entra 35
  VISITA 35
    entra 20
    VISITA 20
      null -> regresa (izquierdo)
      entra 28
      VISITA 28
        null -> regresa (izquierdo)
        null -> regresa (derecho)
    entra 45
    VISITA 45
      entra 40
      VISITA 40
        null -> regresa (izquierdo)
        null -> regresa (derecho)
      null -> regresa (derecho)
  entra 80
  VISITA 80
    entra 70
    VISITA 70
      null -> regresa (izquierdo)
      entra 75
      VISITA 75
        null -> regresa (izquierdo)
        entra 78
        VISITA 78
          null -> regresa (izquierdo)
          null -> regresa (derecho)
    entra 90
    VISITA 90
      entra 87
      VISITA 87
        null -> regresa (izquierdo)
        null -> regresa (derecho)
Secuencia PREORDEN: 60 35 20 28 45 40 80 70 75 78 90 87
```

Tres eliminaciones (hoja, un hijo y dos hijos):

```
=== 4. Eliminacion 1: hoja (28) ===
Preorden: 60 35 20 45 40 80 70 75 78 90 87


=== 5. Eliminacion 2: nodo con un hijo (75, tiene a 78 a la derecha) ===
Preorden: 60 35 20 45 40 80 70 78 90 87


=== 6. Eliminacion 3: nodo con dos hijos (60, raiz) ===
Preorden: 70 35 20 45 40 80 78 90 87
Inorden:  20 35 40 45 70 78 80 87 90
```

El detalle completo de la resolución a mano (códigos personales, árboles en Mermaid y comparación punto por punto) está en el archivo TRAZAS.md del repositorio.

15. Casos de prueba obligatorios y su resultado

```
[OK]   Biblioteca solo con la raiz: altura 0
[OK]   Biblioteca solo con la raiz: 1 hoja
[OK]   Biblioteca inicial: altura 2
[OK]   Biblioteca inicial: 4 hojas
[OK]   Biblioteca inicial: 13 libros
[OK]   Nivel de BIB = 0
```

```
[OK]   Nivel de sw (minusculas) = 1
[OK]   Nivel de SW-E2 = 2
[OK]   Nivel de XYZ = -1
[OK]   Agregar ESTANTE como hijo de BIBLIOTECA = false
[OK]   Preorden SW-E1 = 60 35 20 45 40 80 70 75 90
[OK]   Inorden SW-E1 = 20 35 40 45 60 70 75 80 90
[OK]   Posorden SW-E1 = 20 40 45 35 75 70 90 80 60
[OK]   Insertar codigo repetido 60 = false
[OK]   caminoBusqueda(42) = [60, 35, 45, 40]
[OK]   contarEnRango(40, 75) = 5
[OK]   Tras insertar 42 y eliminar 20,70,60: preorden = 75 35 45 40 42 80 90
[OK]   Buscar codigo inexistente = null
[OK]   Modificar codigo inexistente = false
[OK]   Eliminar codigo inexistente = false
[OK]   Libro con codigo invalido lanza IllegalArgumentException
[OK]   Libro con anio fuera de 1900-2026 lanza IllegalArgumentException
[OK]   esABBValido() con nieto mal ubicado = false
[OK]   esABBValido() en el catalogo correcto = true
[OK]   Extra: desdePreorden reconstruye el mismo recorrido inorden
==================================================
25/25 pruebas OK
```

16. Preguntas de reflexión

• ¿Por qué buscar en el ABB es O(h) y en el árbol general es O(N)? ¿Qué le pasa a h si se insertan los códigos ya ordenados?

En el ABB cada comparación descarta un subárbol completo (se continúa solo a la izquierda o a la derecha), por lo que la búsqueda visita a lo sumo un camino de la raíz a una hoja: O(h). En el árbol general los hijos no tienen ningún orden, así que hay que revisarlos todos en el peor caso: O(N). Si los códigos se insertan ya ordenados, cada nodo nuevo se convierte en hijo derecho del anterior y el árbol degenera en una lista: h = N y la búsqueda pasa a O(N).

• ¿Por qué, al eliminar un nodo con dos hijos, se usa el sucesor inorden y no cualquier nodo del subárbol derecho?

Porque el sucesor inorden (el mínimo del subárbol derecho) es la clave más pequeña que es mayor que el nodo eliminado: es mayor que todos los del subárbol izquierdo y menor que todos los restantes del derecho, de modo que la propiedad del ABB se conserva. Si se usara cualquier otro nodo del subárbol derecho, un antecesor o sucesor quedaría mal ubicado y el árbol dejaría de ser un ABB válido.

• ¿Cómo podría utilizarse un árbol general o un ABB en el proyecto integrador SmartETL para organizar o buscar registros?

El árbol general serviría para modelar la jerarquía del proceso ETL (proyecto → flujos → transformaciones, o fuentes → procesos → destinos), de la misma forma que Biblioteca → Área → Estante. Un ABB permitiría indexar los registros procesados por una clave numérica (id, timestamp) para búsquedas, inserciones y consultas por rango eficientes en O(h), por ejemplo para localizar registros entre dos fechas sin recorrer toda la colección.

2.7 Resultados obtenidos

Se implementó una aplicación Java completamente funcional que integra un árbol general n-ario y varios ABB, obteniendo 25/25 pruebas OK, incluidos todos los casos de prueba obligatorios del deber y el extra desdePreorden. Se comprobó que el inorden de un ABB siempre devuelve los

códigos ordenados y que la traza del preorden permite visualizar el orden real de las llamadas recursivas.

Logramos representar la jerarquía de la biblioteca mediante enumeraciones con reglas de negocio (puedeTenerHijo) y validaciones en el modelo, evitando estados inválidos desde la construcción de los objetos.

Implementamos la eliminación de los tres casos del ABB (hoja, un hijo y dos hijos con sucesor inorden) y la verificamos tanto en la prueba de escritorio como en la ejecución del programa, coincidiendo ambos resultados.

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo ☐ Trabajo en equipo ☐ Comunicación asertiva ☐ La empatía

☒ Pensamiento crítico ☐ Flexibilidad ☒ La resolución de conflictos ☒ Responsabilidad

2.9 Conclusiones

Concluimos que el ABB reduce el costo de búsqueda a O(h) gracias al orden de sus claves, mientras que el árbol general es la estructura natural para representar jerarquías sin orden como la de la biblioteca; ambas se complementan en una misma aplicación.

Comprendimos que la recursividad permite implementar de forma concisa y correcta todas las operaciones de los árboles (inserción, búsqueda, recorridos, eliminación y retos), siempre que se definan bien los casos base.

Verificamos que, si los datos se insertan ordenados, el ABB degenera en una lista con h = N, por lo que en la práctica el balanceo es importante para conservar el rendimiento.

2.10 Recomendaciones

Resolver primero la prueba de escritorio (inserciones, recorridos y eliminaciones a mano) y luego compararla con la salida del programa, ya que facilita detectar errores de comprensión antes de programar.

Respetar desde el inicio los nombres de paquetes, clases y métodos exigidos por el deber y realizar commits frecuentes y significativos para evidenciar el desarrollo individual.

2.11 Referencias bibliográficas

[1] J. Caiza, Estructuras de datos no lineales: árboles generales y árboles binarios de búsqueda; recorridos; eliminación con sucesor inorden., Universidad Técnica de Ambato, 2026.

[2] Deber individual APE 3: Catálogo de la Biblioteca FISEI con árbol general y ABB en Java, Universidad Técnica de Ambato, 2026.

2.12 Anexos

GitHub del Deber: https://github.com/jullisaaltamirano2017-boop/Deber-Arboles-AltamiranoJullisa

Contenido del repositorio: código fuente en src/ (enums, modelo, negocio, app), README.md con el problema y las instrucciones de ejecución, TRAZAS.md resuelto a mano con los árboles en Mermaid, y la evidencia 25/25 pruebas OK de PruebasBiblioteca.